

Software Engineering

Unit III

In-Depth Study Notes

Software Design | Quality Management | Software Implementation

BCA-204 (4th Sem) and BCA-108T

Covers every syllabus topic of Unit III and answers the questions asked in Jul-2023, Jun-2024 and May-Jun-2025 papers.

Contents

Part A: Software Design	3
1. Introduction to Software Design	3
2. Modules and Modularity	4
3. Cohesion	5
4. Coupling	9
5. Cohesion versus Coupling	11
Part B: Quality Management	13
6. Quality Concept	13
7. Key Components of Quality Management	14
8. Software Quality Assurance (SQA)	15
9. Total Quality Management (TQM)	17
10. Software Review	19
11. Software Inspection	20
Part C: Software Implementation	22
12. Structured Coding Techniques	22
13. Coding Style	23
14. Coding Standards and Guidelines	25
15. Documentation Guidelines	26
16. Reverse Engineering	27
17. Software Re-engineering	29
18. Configuration Management	31
Part D: Exam Preparation	35
19. Previous-Year Questions with Answer Outlines	35
20. Likely Questions from Topics Not Yet Asked	36
21. One-Page Revision Summary	36
Appendix: Asked in Papers but Outside the Syllabus List	37
A. CMM versus ISO 9000 (108T Q7b, 5 marks)	37
B. Object-oriented terms (Jun-25 Q7b, 5 marks)	37

Part A: Software Design

This part covers the syllabus line **Software Design: Cohesion and Coupling, Classification of Cohesiveness and Coupling**. Before cohesion and coupling can be understood properly, we need to know what design is, why we break software into modules, and what makes a design good. All of that is covered first.

1. Introduction to Software Design

1.1 What is design?

Software design is the activity that turns the *what* of the Software Requirements Specification (SRS) into the *how* of a solution. The SRS says what the system must do; design decides how the system will be organised so that it does it. It is the bridge between requirements analysis and coding, and the output of design is a blueprint that programmers can implement directly.

Design is the first step in the development phase where we move from the problem domain to the solution domain. Mistakes found at design are many times cheaper to fix than mistakes found after the product has been delivered, which is why design deserves careful attention.

1.2 Conceptual design and technical design

The Jun-2025 paper asks for the difference between conceptual design and technical design. Design is often carried out in two stages.

Basis	Conceptual design (logical design)	Technical design (physical design)
Question answered	What will the system do?	How will the system be built?
Audience	Customer, user, management (non-technical)	Programmers, testers, system engineers
Language used	Plain language, simple diagrams, screen layouts	Technical diagrams, algorithms, data structures, database schema
Contents	Functions, major components, inputs and outputs, data flow, overall architecture	Module structure, interfaces between modules, file and table design, algorithms, hardware and software platform
Dependence on technology	Independent of technology	Depends on chosen language, DBMS, hardware
Output	Conceptual (logical) design document accepted by the customer	Detailed (physical) design document used for coding
Example	"The library system will let a member search, issue and return books."	"Issue_Book() module takes Member_ID and Book_ID, checks the Member table, and updates the Issue table."

1.3 Levels of design

- **Architectural (high-level) design:** identifies the major modules (sub-systems), what each does, and how they are related and communicate.
- **Detailed (low-level) design:** specifies the internals of each module: algorithms, data structures, and interface details.
- **Interface and data design:** defines how modules pass data to each other, and how data is stored.

1.4 Basic design principles

- **Abstraction:** concentrate on essential features and hide details. Procedural abstraction hides how a function works behind its name and parameters; data abstraction hides how data is represented.
- **Modularity:** divide the system into separate named modules that are developed and tested independently.
- **Information hiding:** a module keeps its internal data and decisions private and exposes only a small, stable interface. A change inside the module then does not affect other modules.
- **Stepwise refinement (top-down design):** start from a general statement of the solution and refine it step by step into finer detail.
- **Functional independence:** modules should do one specific thing and have little interaction with others. This is measured with **cohesion** and **coupling**, which are the core of this part.
- **Reusability:** design modules so they can be used in other places.

1.5 Characteristics of a good design

- **Correctness:** it implements every requirement in the SRS.
- **Understandability:** easy for others to understand, because maintenance needs this.
- **Efficiency:** uses time and memory sensibly.
- **Maintainability:** a change should affect only a small part of the system.
- **Modularity with high cohesion and low coupling.**

2. Modules and Modularity

2.1 What is a module?

A **software module** is a self-contained, separately named unit of a program that performs a defined function and has a clear interface to other modules. In practice a module may be a function, procedure, subroutine, class, or a file of related functions. A well-designed module can be compiled, tested and understood on its own. (The Jun-2024 question "What is a software module and cohesion?" starts from this definition.)

2.2 Why divide software into modules?

- **Manage complexity:** a large problem becomes a set of small problems. The human mind can handle only a limited amount of detail at once.
- **Parallel development:** different programmers can build different modules at the same time.
- **Easy testing and debugging:** a module can be tested in isolation (unit testing), and errors are localised.
- **Easy maintenance:** a change is confined to a few modules.
- **Reuse:** a tested module can be reused in other systems.

2.3 How big should a module be?

If a system is divided into too few modules, each module becomes large and hard to understand, so the cost of each module is high. If it is divided into too many modules, the cost of integrating them and managing the interfaces becomes high. Total cost is lowest at some middle number of modules. The aim is therefore to find the right balance and not simply to make modules as small as possible.

2.4 Terms used in a module hierarchy (structure chart)

Term	Meaning
Depth	Number of levels of control in the module hierarchy.
Width	Largest number of modules at any one level.
Fan-out	Number of modules directly called (controlled) by a module. Very high fan-out suggests the module has too much control.
Fan-in	Number of modules that directly call (use) a given module. High fan-in shows good reuse.
Superordinate / subordinate	A module that controls another is superordinate; the controlled module is subordinate.

Key Point: A good structure has **low fan-out** (a module should not control too many others) and **high fan-in** for common utility modules (they are reused).

3. Cohesion

3.1 Definition

Cohesion is a measure of the functional strength of a module. It describes how closely the elements inside one module (statements, data items, function calls) are related to one another and how much they contribute to one single purpose. Cohesion is an **intra-module** (inside the module) property.

A module with **high cohesion** does one job and does it completely. A module with **low cohesion** is a collection of unrelated activities. We want high cohesion because it makes modules easier to understand, test, reuse and maintain, and it normally leads to low coupling as well.

Quick Test: Test for cohesion: try to describe the purpose of the module in one simple sentence. If you need the words "and", "then", or a list of unrelated verbs, cohesion is low. If a single verb-object phrase (for example "compute salary") is enough, cohesion is high.

3.2 Classification of cohesion (weakest to strongest)

Cohesion is classified into seven levels, proposed by Constantine and Yourdon. Going down the list, cohesion becomes stronger and therefore better.

Rank	Type	Strength
1	Coincidental	Worst (lowest cohesion)
2	Logical	Very low
3	Temporal	Low
4	Procedural	Medium
5	Communicational	Good
6	Sequential	Very good
7	Functional	Best (highest cohesion)

3.2.1 Coincidental cohesion

The elements of the module have **no meaningful relationship**. They are placed together by chance, often because someone split a large program into pieces arbitrarily, or because several unrelated small tasks were thrown into a "utility" module. This is the weakest form.

```
Module: MISC_ROUTINES
- print_report()
- convert_date_format()
- calculate_tax()
- clear_screen()
```

Problems: the module cannot be described in one sentence, it is hard to reuse (we get unwanted functions along with the one we need), and a change in one function risks breaking others. **Remedy:** split the module so that each function sits with related functions.

3.2.2 Logical cohesion

The module performs a **set of similar or logically related operations**, and the caller chooses which one to run by passing a control flag. The elements are in the same logical category (for example "all input operations"), but they do different things.

```
void io_module(int device, data) {
    if (device == 1) read_from_keyboard();
    else if (device == 2) read_from_file();
    else if (device == 3) read_from_network();
}
```

Problems: the interface is complicated by the control flag, the code of different operations is tangled together, and part of the module is unused in any one call. It also leads to **control coupling**.

Remedy: create a separate module for each operation.

3.2.3 Temporal cohesion

The elements are grouped because they must be **executed at the same time** (in the same phase of execution), not because they are related in function. Typical examples are initialisation and termination routines.

```
void initialise_system() {  
    open_files();  
    clear_counters();  
    set_default_values();  
    connect_to_database();  
}
```

Problems: the activities are unrelated to each other except in timing. A change in one activity (for example the database) can force changes to the whole module. It is slightly better than logical cohesion because it is clear when the code runs, and the module is easy to call. **Remedy:** make the init routine only call separate cohesive modules.

3.2.4 Procedural cohesion

The elements belong to a **fixed sequence of steps** (a procedure) in which **control flows from one element to the next**, but the elements do not share data and do not contribute to a single function. They are connected only by the order of execution.

```
void process_request() {  
    check_user_permission();  
    open_account_file();  
    write_log_entry();  
    display_menu();  
}
```

Difference from sequential cohesion: in procedural cohesion only *control* passes from step to step; in sequential cohesion the *output data* of one step becomes the input of the next.

3.2.5 Communicational cohesion

All elements of the module operate on the **same input data or produce the same output data**. They are grouped because they work on the same data structure, though the order of their execution does not matter.

```
void process_student_record(Student s) {  
    calculate_total_marks(s);  
    calculate_percentage(s);  
    print_marksheet(s);  
    update_student_file(s);  
}
```

Problem: the functions are not independent in purpose, so the module is harder to reuse; a change in the data format affects all of them. It is a reasonably good level, and often acceptable.

3.2.6 Sequential cohesion

The elements are linked so that the **output of one element is the input of the next**, like an assembly line or pipeline. Each part is a step toward a common goal.

```
void prepare_report() {
    raw  = read_data();
    clean = validate(raw);
    final = format(clean);
    print(final);
}
```

It is good cohesion because the data flows naturally, but it often cannot be called functional because the module performs several steps rather than one complete function, and reusing a single step on its own is not possible.

3.2.7 Functional cohesion (best)

Every element of the module contributes to the **execution of one and only one well-defined task**. The module does exactly one job and does all of it. It can be named with a single verb-object phrase.

```
float compute_net_salary(float basic, float da, float tax) {
    return basic + da - tax;
}
/* other examples: calculate_square_root(), print_invoice(),
   search_book(), validate_password() */
```

Advantages: easy to understand, can be reused wherever the function is needed, easy to test, and changes stay inside the module. This is the target of good design.

3.3 Summary table of cohesion types

Type	Elements are related by	Example
Coincidental	Nothing (chance)	Misc/utility module with unrelated functions
Logical	Same category of work, selected by a flag	One routine for all input devices
Temporal	Executed at the same time	Initialisation / shutdown routine
Procedural	Fixed order of execution (control passes)	Check permission, open file, write log
Communicational	Same input or output data	Calculate and print marks of one student record
Sequential	Output of one is input of next	Read, validate, format, print
Functional	One single task	Compute_salary()

3.4 Advantages of high cohesion

- Module is easier to understand and name.
- Fewer errors; testing is simple.
- Reuse is easy since the module has a single purpose.
- Maintenance is easy because a change request normally affects one module.
- Leads to low coupling between modules.

Exam Tip: For the 12.5 mark question "What is a software module and cohesion? Discuss types of module cohesion", write: definition of module, definition of cohesion, ranking diagram (coincidental to functional), one paragraph plus a small example for each of the seven types, and the summary table.

4. Coupling

4.1 Definition

Coupling is a measure of the **degree of interdependence between modules**. It tells us how much one module relies on the internal details of another and how strongly a change in one module will affect others. Coupling is an **inter-module** (between modules) property.

We want **low (loose) coupling**: modules that are as independent as possible, so that a module can be understood, tested, changed and reused without touching other modules. With **high (tight) coupling**, modules depend heavily on each other, so a change ripples through the system and errors spread.

4.2 Factors that decide the degree of coupling

- **Type of connection:** whether modules communicate through parameters or by sharing global data or internal code.
- **Amount of data** passed between modules.
- **Complexity of the interface:** a simple interface means lower coupling.
- **What is passed:** data only, or control information as well.
- **Number of connections** between the two modules.

4.3 Classification of coupling (worst to best)

Rank	Type	Strength
1	Content (pathological)	Worst (highest coupling)
2	Common	Very high
3	External	High
4	Control	Medium
5	Stamp (data-structured)	Low
6	Data	Best (lowest coupling)

Some books add a seventh level, **no coupling** (completely independent modules), which is ideal but usually not possible in a working system because modules have to cooperate.

4.3.1 Content coupling

One module **directly uses or changes the internal data or code** of another module. For example, module A branches into the middle of module B, or modifies a local variable or instruction of B. It is also called *pathological coupling*. It happens mostly in assembly language, or in languages that allow jumps into other routines.

```
Module A: GOTO label_inside_Module_B;    /* jumps into B */
Module A: changes a local variable of B
```

Problem: any change inside B breaks A. Modules cannot be understood separately. This should be avoided completely.

4.3.2 Common coupling

Two or more modules **share the same global data** (a common data area, global variables, or a shared common block). Each module can read and write the shared data.

```
int balance;                                /* global variable */
void deposit() { balance = balance + amt; }
void withdraw() { balance = balance - amt; }
void audit()   { if (balance < 0) alert(); }
```

Problems: a bug or change in any module can corrupt data used by all others; it is hard to find which module changed the data; modules cannot be reused independently; and testing is difficult. **Remedy:** pass the needed data as parameters.

4.3.3 External coupling

Modules are tied to an **externally imposed format, protocol, device or interface**. They share the same file format, communication protocol or hardware device, not the data directly. Example: several modules all depend on the same file layout or on a particular I/O device.

Problem: if the external format or device changes, all modules that rely on it must change. Limit the number of modules that know about the external element (for example by keeping all device access in one module).

4.3.4 Control coupling

One module **passes control information** (a flag, switch, or code) that decides the **internal logic of the other module**. The caller must know how the called module works inside.

```
print_report(report_data, flag);
/* flag = 1: print summary, flag = 2: print details, flag = 3: print graph */
```

Problems: the caller needs to know the internals of the called module; the called module has several jobs (it often has logical cohesion). **Remedy:** split into separate modules, one per action.

4.3.5 Stamp coupling (data-structured coupling)

Modules communicate by passing a **composite data structure** (a record, array or object) while the called module **uses only part of it**. Both modules must know the structure of the data.

```
float get_age_group(Student s); /* receives whole record */
/* but uses only s.date_of_birth */
```

Problems: the called module can access more data than it needs, and a change in the structure affects every module that uses it. **Remedy:** pass only the fields actually required.

4.3.6 Data coupling (best)

Modules communicate only by passing **simple, necessary data items** as parameters (integers, characters, floats), and every parameter is actually used. There is no shared global data and no control information.

```
float area = compute_area(length, width);
float tax  = calculate_tax(income, rate);
```

Advantages: the interface is minimal and clear, modules are independent, easy to test, easy to change and reuse. This is the best type of coupling that a practical design can achieve.

Exam Answer: Which coupling is best and why? Data coupling is the best (lowest) because modules exchange only the minimum, necessary, simple data through parameters. There are no hidden links, so a change inside a module does not spread to others and each module can be tested and reused alone. Content coupling is the worst.

4.4 Summary table of coupling types

Type	How modules are connected	Level
Content	One module uses internal code or data of another	Highest (worst)
Common	Share global data	Very high
External	Share an external file format, protocol or device	High
Control	One passes a flag that controls the other's logic	Medium
Stamp	Pass a whole data structure, use part of it	Low
Data	Pass only needed simple parameters	Lowest (best)

4.5 How to reduce coupling

- Pass data through parameters, not global variables.
- Pass only the data that is needed (prefer simple items to whole records).
- Avoid passing control flags; split the module instead.
- Use information hiding, and keep module interfaces small and well-defined.
- Keep access to external devices and file formats in one module.
- Never jump into or modify the code of another module.

5. Cohesion versus Coupling

Basis	Cohesion	Coupling
Meaning	Strength of relationship of elements inside a module	Degree of dependence between modules
Scope	Intra-module (within one module)	Inter-module (between modules)
Goal	Should be as high as possible	Should be as low as possible
Best type	Functional	Data

Basis	Cohesion	Coupling
Worst type	Coincidental	Content
Number of types	7	6 (plus "no coupling" in some books)
Effect of improving	Module easier to understand and reuse	Module easier to change, test and replace
Relation	High cohesion usually gives low coupling and vice versa	

Key Point: Good design means **high cohesion and low coupling**. The two are linked: when each module does one job (functional cohesion), it rarely needs anything beyond the data given to it (data coupling).

Exam Tip: A frequently asked pairing is "Explain different types of coupling" (6 marks, Jul-23; 5 marks, Jun-25) and "Explain different types of cohesion" (6 marks). Draw the ordered scale first, then explain each type with a definition and a one-line example. Mention the best and worst type.

Part B: Quality Management

This part covers **Quality concept, Software Quality Assurance (SQA), Total Quality Management (TQM), software review and software inspection.**

6. Quality Concept

6.1 What is quality?

Quality has been defined in several ways, and all of them are useful in exams.

- **Fitness for use** (Juran): the product satisfies the needs of its user.
- **Conformance to requirements** (Crosby): the product matches the specified requirements. Quality is "free" because preventing defects costs less than fixing them.
- **ISO definition:** the totality of features and characteristics of a product that bear on its ability to satisfy stated or implied needs.

Software quality therefore means that the software meets its functional and non-functional requirements, follows documented development standards, and also meets the implicit expectations of users (such as ease of use and reliability). Software is intangible, so quality cannot be seen directly; it is judged through its attributes, and it is built into the product during development, not added at the end by testing.

6.2 Two views: quality of design and quality of conformance

- **Quality of design:** the characteristics that designers specify for a product: grade of materials, tolerances, performance. In software it is how well the design and the specification match what the user needs.
- **Quality of conformance:** the degree to which the implementation follows the design and the specification. High conformance gives a product that does what the design says.

6.3 Software quality factors (McCall)

McCall classified quality into **11 factors** grouped around three aspects of a software product.

Aspect	Factor	Meaning
Product operation (how it runs)	Correctness	Does what the specification requires
	Reliability	Performs its function without failure over time
	Efficiency	Uses minimal CPU time, memory and resources
	Integrity	Controls unauthorised access to software and data (security)
	Usability	Effort needed to learn and operate it
Product revision (how easily it changes)	Maintainability	Effort to find and fix errors
	Flexibility	Effort to modify the operating system or add features

Aspect	Factor	Meaning
	Testability	Effort to test that the software performs its function
Product transition (how it adapts)	Portability	Effort to move it to another platform
	Reusability	Extent to which parts can be used in other applications
	Interoperability	Effort to couple it with another system

Other well-known models are Boehm's model and the ISO/IEC 9126 model, which lists functionality, reliability, usability, efficiency, maintainability and portability as the main characteristics.

6.4 Cost of quality

Quality has a cost, and failures cost more. The cost of quality is divided into four parts.

Cost	Meaning	Example
Prevention cost	Spent to stop defects occurring	Training, standards, planning, tools
Appraisal cost	Spent to find defects	Reviews, inspections, testing
Internal failure cost	Defects found before delivery	Rework, retesting, debugging
External failure cost	Defects found after delivery	Complaints, repairs, warranty, loss of reputation

Key Point: The later a defect is found, the more it costs to fix. Spending on prevention and appraisal reduces the larger failure costs.

7. Key Components of Quality Management

Quality management is the set of activities that direct and control an organisation with regard to quality. In software engineering it has four key components (the Jun-2025 question on "key components of quality management").

7.1 Quality planning

Deciding which quality goals the project will aim for and how to reach them. It identifies the standards to be used, the procedures, the reviews and tests, the metrics, the tools and the responsibilities, and records them in a **quality plan**.

7.2 Quality assurance (QA)

A **process-oriented**, preventive activity. It makes sure that the right processes and standards are defined and followed so that the product will be of good quality. It includes establishing standards, process audits and reviews of work products.

7.3 Quality control (QC)

A **product-oriented**, detective activity. It examines the actual products (code, documents) through reviews, inspections and testing to *find* defects, and checks whether they meet the standards. Defects found are corrected and the results reported.

7.4 Quality improvement

Collecting and analysing defect and process data (metrics) to find the causes of defects and improving the process so that they do not recur. It supports continuous improvement and is the link to TQM.

Basis	Quality Assurance (QA)	Quality Control (QC)
Focus	Process	Product
Aim	Prevent defects	Detect and correct defects
Nature	Proactive	Reactive
When	Throughout the life cycle	After a work product is produced
Activities	Standards, audits, process definition, training	Reviews, inspection, testing
Who	Quality assurance group / whole team	Testing team and reviewers

8. Software Quality Assurance (SQA)

8.1 Definition

Software Quality Assurance is a planned and systematic set of activities, applied across the whole software process, that provides confidence that the software product and the process used to make it conform to the defined requirements and standards. SQA covers both *process* and *product*, starts at the requirements stage and continues until maintenance.

8.2 Objectives of SQA

- Prevent defects by improving the process.
- Make sure the software meets the specified requirements.
- Make sure documented standards and procedures are followed.
- Make the project status and problems visible to management.
- Give the customer confidence in the product.

8.3 How software quality is determined

The Jun-2025 (108T) paper asks "How do we determine software quality assurance?". Quality is determined by measuring and checking the product and the process against agreed criteria:

- Checking conformance to requirements and to standards.
- Software quality factors (correctness, reliability, usability, maintainability and the rest).
- Metrics such as defect density, number of defects found per review, mean time to failure, and complexity measures.
- Results of reviews, inspections, audits and testing.
- Customer feedback and defect reports after delivery.

8.4 The SQA group

Many organisations have an **independent SQA group** that works on behalf of the customer and management. It is independent so that it can report problems honestly, without pressure from the development schedule. It does not build the product; it watches the process and the work products and informs senior management.

8.5 SQA activities

The usual SQA activities are as follows.

Activity	Description
1. Prepare an SQA plan	The plan covers the evaluations to be done, audits and reviews, standards, procedures for error reporting, documents to be produced, and feedback to the team.
2. Participate in process description	Review the software process chosen by the project to make sure it fits organisational policy and standards.
3. Review software engineering activities	Check each activity against the defined process, and note deviations.
4. Audit designated work products	Check selected work products for conformance to the standard, and review the results.
5. Make sure deviations are documented and handled	Any departure from the process or standards is recorded and followed up in line with procedure.
6. Record non-compliance and report	Non-compliance is reported to senior management and tracked until resolved.
7. Collect and analyse metrics	Gather defect and process data to find trends and to improve.
8. Oversee testing and change control	Check that testing is planned and executed properly and that changes follow configuration management.

8.6 Contents of an SQA plan (IEEE 730 style)

- Purpose and scope of the plan.
- Reference documents and the management structure.
- Documentation to be produced and standards, practices and conventions to be followed.
- Reviews and audits (what, when, by whom).
- Testing activities.
- Problem reporting and corrective action.
- Tools, techniques and methodologies, and records to be kept.
- Training and risk management.

8.7 Techniques used by SQA

- Technical reviews, walkthroughs and inspections.
- Audits of process and product.
- Software testing and its review.

- Metrics collection and analysis; statistical quality assurance (finding the vital few causes that produce most defects).
- Enforcement of standards (for example through checklists and tools).

Exam Tip: For "What is Quality in Software? How do we determine SQA? Explain various SQA activities" (10 marks), cover: definition of quality, McCall factors, how quality is determined, definition and objectives of SQA, and the list of activities in section 8.5.

9. Total Quality Management (TQM)

9.1 Definition

Total Quality Management is an organisation-wide management approach in which **every employee at every level** takes responsibility for quality, with the focus on **customer satisfaction** and **continuous improvement** of products, services and processes. It began in manufacturing (W. Edwards Deming, Joseph Juran, and Japanese industry) and is applied to software organisations.

9.2 The word "Total"

Quality is not the job of a quality department only. "Total" means that all functions of the organisation (management, development, testing, sales, support) and all people take part, and that suppliers and customers are included.

9.3 Principles of TQM

Principle	Explanation
Customer focus	Quality is decided by the customer. Find and meet customer needs and expectations.
Total involvement	Everyone is responsible for quality; teamwork and training are encouraged.
Process approach	Results come from processes. Improve the process, not blame the person.
Continuous improvement	Never be satisfied; keep improving using the PDCA cycle (Plan, Do, Check, Act). The Japanese term is Kaizen.
Decision making based on data	Measure and analyse facts, metrics and defect data before deciding.
Strong leadership	Top management sets quality policy and gives direction and resources.
Prevention not detection	Build quality in rather than inspect it in at the end.
Supplier relationships	Work with suppliers as partners for quality inputs.

9.3.1 The PDCA (Deming) cycle

- **Plan:** find the problem and plan the change.
- **Do:** try out the change on a small scale.
- **Check:** study the results, measure against the plan.
- **Act:** adopt the change if it works, otherwise start the cycle again.

9.4 Deming's 14 points (summary)

Deming gave 14 points for management, which form the basis of TQM: create constancy of purpose for improvement; adopt the new philosophy; stop depending on mass inspection; do not award business on price alone; improve constantly the system of production and service; institute training on the job; institute leadership; drive out fear; break down barriers between departments; eliminate slogans and targets for the workforce; eliminate numerical quotas; remove barriers to pride of workmanship; encourage education and self-improvement; and make the transformation everyone's job.

9.5 TQM in software engineering

- Quality is planned from the requirements stage, using prevention (standards, reviews) more than detection (testing).
- Use metrics (defect density, rework effort, customer complaints) to monitor processes and to improve them.
- Teams are trained and share responsibility for the quality of what they build.
- Defects are analysed to find root causes, and processes are changed to remove them.
- Customer feedback drives the next release.

9.6 Benefits and difficulties

Benefits	Difficulties
Fewer defects and less rework	Needs long-term commitment of top management
Higher customer satisfaction	Culture change is slow and may be resisted
Better productivity and lower cost	Training and measurement need investment
Better team spirit and morale	Results take time to show; initial cost is high
Improved process maturity	Difficult to measure quality of intangible software

9.7 SQA versus TQM

Basis	SQA	TQM
Scope	Software process and product	Whole organisation
Responsibility	Mainly the SQA group	Everyone
Focus	Conformance to standards	Customer satisfaction and continuous improvement
Nature	A set of activities within a project	A management philosophy and culture

10. Software Review

10.1 Definition

A **software review** is a systematic examination of a work product (the SRS, design, code, test plan, user manual) by one or more people other than the author, in order to **find defects, check conformance to standards and requirements, and suggest improvements**. Reviews are a **static** verification technique: the program is not run.

10.2 Why review?

- Defects are found early, when they are cheap to fix. A design error found in a review costs far less than the same error found in the field.
- They are effective: many reviews find a large share of the errors that testing would find later, and they find things testing cannot (poor standards, unclear documents).
- They spread knowledge in the team and train juniors.
- They help to ensure uniformity and standards.
- They improve visibility and give management confidence.

10.3 Types of reviews

Type	Description	Formality
Informal review (peer desk check)	Author asks a colleague to look at the work. No records, no fixed procedure.	Low
Walkthrough	Author presents the work product step by step to a group; the audience asks questions and comments. Mainly for understanding and early feedback.	Low to medium
Technical (peer) review	A team of technical peers examine the product against specifications and standards. Findings are recorded.	Medium
Management review	Managers examine project status, schedules and plans.	Medium
Audit	Independent examination for conformance to standards and procedures.	High
Inspection	Most formal, defined roles and phases, uses checklists and data (see next section).	Highest

10.4 The review process

- 1 **Planning:** select the product, participants and the date.
- 2 **Preparation:** reviewers study the material beforehand.
- 3 **Review meeting:** the author presents, reviewers raise issues; a recorder writes them down.
- 4 **Decision:** accept as is, accept with minor changes, or reject and re-review.
- 5 **Rework and follow-up:** the author fixes the problems and the fixes are verified.

10.5 Guidelines for conducting reviews

- Review the **product, not the producer**; keep the tone constructive.
- Set an agenda and keep to it; limit the meeting length (about two hours at most).
- Limit debate; record issues and do not try to solve all of them in the meeting.
- State problems, do not argue for solutions.
- Keep written notes and a defect list.
- Limit the number of participants and insist on advance preparation.
- Use a checklist, and train the reviewers.
- Review the review itself to improve the process.

11. Software Inspection

11.1 Definition

Software inspection is a **formal, rigorous and structured peer review** in which a trained team examines a work product (requirements, design or code) against a checklist and standards in order to find defects **before testing**. It was introduced by Michael Fagan at IBM in the 1970s and is often called **Fagan inspection**. Its main aim is defect detection only; the solution is left to the author.

11.2 Roles in an inspection

Role	Responsibility
Moderator (leader)	Trained person who plans, schedules and runs the inspection and makes sure the process is followed.
Author (producer)	Creator of the work product; answers questions and later fixes the defects. Does not lead or defend.
Reader	Paraphrases or reads the product aloud during the meeting.
Recorder (scribe)	Writes down all defects and decisions.
Inspectors (reviewers)	Study the material in advance and find defects using checklists.

11.3 Phases of inspection (Fagan process)

Phase	What happens
1. Planning	The moderator checks that the product meets entry criteria (for example, that it compiles or the document is complete), chooses the team, and schedules the meeting.
2. Overview	The author briefly explains the product and its background to the team. (Sometimes omitted for small products.)
3. Preparation	Each inspector studies the material alone, using checklists, and notes possible defects.
4. Inspection meeting	The reader goes through the product; inspectors report defects, which the recorder logs. No attempt is made to fix them. The meeting is short (around two hours).
5. Rework	The author corrects all defects found.

Phase	What happens
6. Follow-up	The moderator checks that every defect has been fixed; the inspection is closed (exit criteria met), or re-inspection is ordered if too many changes were made.

11.4 Checklists and metrics

Checklists list the typical errors to look for, and are specific to the work product. For code, they include items such as: Are all variables initialised? Are loops terminated correctly? Are array bounds checked? Are standards followed? Are all error conditions handled?

Metrics collected include the number and type of defects found, the size of the product inspected, preparation and meeting effort, and defect density (defects per KLOC). These are used to improve the process and to predict remaining defects.

11.5 Benefits

- Finds defects very early (before testing), cutting rework cost.
- Improves the quality of the product and the programmer's own skills.
- Gives data for process improvement.
- Spreads knowledge of the product in the team.

11.6 Review, walkthrough and inspection compared

Basis	Walkthrough	Technical review	Inspection
Formality	Informal	Semi-formal	Formal
Led by	Author	Trained leader / peer	Trained moderator
Purpose	Understanding, feedback	Find defects and check suitability	Find defects only
Preparation	Little or none	Some	Required, with checklist
Records	Optional	Review report	Formal defect log and metrics
Defect data used for improvement	Seldom	Sometimes	Always

Key Point: Review versus inspection: every inspection is a review, but not every review is an inspection. An inspection is the formal, checklist-driven, data-driven and most effective kind of review.

Exam Tip: Jul-23 asks "What is software review and software inspection?" (5 marks). Write a definition of each, the difference (formality, roles, checklists, data) and 3 to 4 benefits. A small table of the Fagan phases will earn the extra marks.

Part C: Software Implementation

This part covers **Structured coding techniques, coding style, standards and guidelines, documentation guidelines, reverse engineering, software re-engineering and configuration management**. Implementation (coding) is the phase where the design is converted into a working program in a programming language.

12. Structured Coding Techniques

12.1 Structured programming

Structured programming is a disciplined way of writing programs in which control flow is built from a small set of well-defined constructs, so that the logic is clear from reading the code from top to bottom. It was advocated by Edsger Dijkstra, whose 1968 letter "Go To Statement Considered Harmful" criticised unrestricted use of GOTO.

The **Bohm-Jacopini theorem** (1966) proved that every computable function can be implemented using only three control structures: sequence, selection and iteration. So the GOTO statement is not necessary.

12.2 The three basic constructs

Construct	Meaning	Examples
Sequence	Statements execute one after another in the order written	<code>a = b + c; d = a * 2;</code>
Selection (decision)	One of two or more paths is chosen on a condition	if-else, switch-case
Iteration (repetition)	A block repeats while a condition holds	while, do-while, for

Each construct has **one entry point and one exit point**. Because every block can be viewed as a single box with one way in and one way out, the program can be understood by reading it as nested boxes.

<pre>/* Unstructured (with GOTO) i = 1; loop: if (i > n) goto end; sum = sum + a[i]; i++; goto loop; end:</pre>	<pre>Structured version */ for (i = 1; i <= n; i++) { sum = sum + a[i]; }</pre>
--	---

12.3 Characteristics of structured code

- Uses only sequence, selection and iteration.
- Single entry and single exit for every block (and normally every function).
- No (or very restricted) use of GOTO; exceptions are error exits and breaking out of nested loops where language support is poor.
- Programs are built from small functions or procedures.
- Top-down design and stepwise refinement are used.

- Data is local to the function wherever possible.

12.4 Structured coding techniques and practices

Technique	Explanation
Top-down design and stepwise refinement	Begin with the main function as a sequence of high-level steps; code each step as a function and refine until each is simple.
Modular programming	Divide the program into small functions or modules, each doing one thing (high cohesion) and interacting through parameters (low coupling).
Avoid GOTO	Use structured loops and conditions; the control flow stays visible.
Information hiding and data abstraction	Keep data and implementation details private to the module; expose operations only.
Limit nesting depth	Deeply nested if or loops are hard to read; restructure with early returns (carefully) or helper functions.
Keep functions short	A function should fit on a screen (often 20 to 50 lines) and do one task.
Avoid global variables	Pass data by parameters to keep coupling low.
Defensive programming	Validate inputs, check return values, handle error conditions and use assertions.
Meaningful names and no magic numbers	Use named constants instead of unexplained values.
Code reuse	Use tested library functions and components rather than writing again.
Consistent error handling	Use one clear method (return codes or exceptions) across the program.

12.5 Advantages of structured coding

- Code is easy to read and understand, so maintenance is easier.
- Fewer logical errors and easier debugging.
- Easier to test because each block has one entry and exit.
- Easy to divide work among programmers.
- Higher productivity and reliability.

13. Coding Style

Coding style is the set of conventions that govern how source code is written and laid out: naming, indentation, spacing, comments and organisation. It does not change what the program does, but it changes how easily people read it. Programmers spend much more time reading code than writing it, so a consistent style reduces the cost of maintenance. The aim is that the whole team writes as if one person had written the code.

13.1 Elements of good coding style

Naming

- Use meaningful, descriptive names (`total_marks`, not `x`).
- Follow one convention consistently: `camelCase`, `PascalCase` or `snake_case`.
- Constants in capital letters (`MAX_SIZE`); avoid single-letter names except loop counters.
- Function names are verbs (`calculate_tax`), variable names are nouns.
- Do not use the same name for different things, and avoid confusing names such as `l` and `1` or `O` and `0`.

Layout and indentation

- Indent consistently (commonly 4 spaces); indent the body of each block.
- Use blank lines to separate logical sections of code.
- Keep lines short (about 80 characters).
- Put one statement per line; place braces in the same style everywhere.
- Use spaces around operators to improve readability.

Comments

- Write a **header comment** for each module or function: purpose, author, date, inputs, outputs, and change history.
- Explain **why** the code does something and not what it does if that is obvious from the code.
- Keep comments correct and updated; a wrong comment is worse than none.
- Do not over-comment trivial statements.

Simplicity

- Avoid clever tricks and obscure language features.
- Avoid magic numbers; use named constants.
- Do not write long expressions; split them into several simple statements.
- Avoid deep nesting and very long functions.

```
/* Poor style */
int f(int a,int b){return a*b*0.18;}

/* Good style */
/* Compute GST amount on price*quantity */
#define GST_RATE 0.18
float calculate_gst(int price, int quantity)
{
    float amount = price * quantity;
    return amount * GST_RATE;
}
```

14. Coding Standards and Guidelines

14.1 Coding standards

Coding standards are **mandatory rules** that every programmer in the organisation must follow while writing code. Compliance is checked in code reviews and inspections (and by automated tools). They give uniformity across projects and teams.

Typical coding standards:

- **Naming rules:** for example, global variables start with a capital letter, local variables are lowercase, constants are all capitals.
- **File headers:** every source file starts with a header giving module name, date of creation, author, modification history, and a summary of purpose and global variables used.
- **Limits:** maximum function length (for example 10 to 50 lines) and maximum number of parameters.
- **Prohibited features:** do not use GOTO; do not use global variables unnecessarily; do not use the same identifier for different purposes; avoid complex conditional expressions.
- **Return conventions:** every function has a single, clear exit and returns a defined error code on failure.
- **Comment rules:** each function carries a comment block, and non-obvious code is explained.
- **Error handling:** every error return value must be checked.

14.2 Coding guidelines

Coding guidelines are **general suggestions** on good practice. They are not compulsory and leave judgement to the programmer, but give direction. Typical guidelines:

- Do not use a coding trick that makes the code obscure; write code that is simple and clear.
- Avoid deep nesting of conditional and loop statements.
- Use meaningful variable and function names that suggest their purpose.
- Do not use an identifier for multiple purposes.
- The code should be well documented; there should be roughly one comment line for about every three source lines.
- Avoid unnecessary complexity and very long functions.
- Prefer standard library functions over home-made equivalents.
- Make the code portable: avoid dependence on a particular compiler or machine.

14.3 Standards versus guidelines

Basis	Coding standards	Coding guidelines
Nature	Mandatory rules	Recommended practices
Compliance	Must be followed, checked in reviews	Programmer may use judgement

Basis	Coding standards	Coding guidelines
Specificity	Precise and measurable (for example naming format, function size limit)	General and broad
Enforcement	Code reviews, inspections, automated checkers	Education and peer encouragement
Example	"Every function header must list parameters."	"Avoid deeply nested loops."

14.4 Benefits of standards and guidelines

- A uniform appearance for code written by many people.
- Easier to understand, review and maintain; new people become productive faster.
- Fewer errors (risky constructs are banned).
- Encourages good practice and makes code review easier to carry out.
- Supports reuse and portability.

14.5 Enforcement

Standards are enforced through **code reviews and inspections**, **static analysis tools and linters** that check style and risky constructs automatically, and **checklists** used by SQA. Organisations often adopt a public standard (for example MISRA C for safety-critical systems, or PEP 8 for Python).

15. Documentation Guidelines

Software documentation is all the written or recorded information that describes the software, how it was built and how it is used. Good documentation is part of the product and is needed for maintenance, training and reuse. It is divided into **internal** and **external** documentation.

15.1 Internal documentation

Documentation that is **inside the source code**. It consists of:

- **Header comments** for each module: name, purpose, author, date, input and output parameters, global variables, change history.
- **Inline comments** that explain difficult or important statements.
- **Meaningful names** for variables and functions (self-documenting code).
- **Good layout**: indentation and spacing.
- Use of **named constants** and a **structured** program.

```

/*****
 * Function : compute_net_salary
 * Purpose  : Computes net salary of an employee
 * Input    : basic, da, tax (all float)
 * Output   : net salary (float)
 * Author   : A. Kumar           Date: 10-Aug-2026
 * Changes  : 15-Aug-2026 - added tax rebate check
 *****/

```

15.2 External documentation

Documents that are **separate from the code** and are produced during the life cycle:

Document	Purpose
SRS (Software Requirements Specification)	Describes what the system must do.
Design document	Architecture, module structure, data design, algorithms, interfaces.
Test plan and test cases	What is tested, how, and the expected results.
User manual	Instructions for end users.
Installation and operations guide	How to install, configure and operate the system.
Maintenance / programmer's guide	Information for those who must modify the code later.
Project plan and quality plan	Schedule, cost, resources, and quality procedures.
Change records / release notes	History of changes and versions.

15.3 Documentation guidelines (good practice)

- Documents must be **accurate, complete** and **up to date** with the code. Update the documentation whenever the code changes.
- Use **standard templates** and a consistent format for all documents.
- Write for the **intended reader** (a user manual should avoid technical jargon).
- Be **clear and concise**; use diagrams, tables and examples.
- Give each document an identifier and version number and place it under **configuration management**.
- Document at the time of development, not after, and review documents like code.
- Document assumptions, decisions and the reasons for them.
- Make the documentation easy to find and search.

Key Point: Documentation is a deliverable. Poor documentation is one of the main reasons maintenance is costly, and it is what reverse engineering has to recover later.

16. Reverse Engineering

16.1 Definition

Reverse engineering is the process of **analysing an existing system** (often only its source or executable code, without documentation) to **identify its components and their relationships and to recreate its design and specification** in another form or at a higher level of abstraction. It moves **backwards** through the life cycle: from code to design to requirements. Chikofsky and Cross defined it in these terms.

Key Point: Reverse engineering **does not change** the software. It only analyses it and produces information (models, diagrams, documents) about it.

16.2 Why is reverse engineering needed?

- Documentation is missing, incomplete or out of date, and the original developers have left.
- To understand a legacy system before maintaining or modifying it.
- To prepare for re-engineering or migration to a new platform.
- To recover lost design and specification.
- To find reusable components.
- To find security flaws or check for malicious code.
- To find and remove errors when no source is available (for example understanding a compiled component).

16.3 Levels of abstraction and characteristics

The result of reverse engineering is described by:

- **Abstraction level:** how sophisticated the recovered information is. It can range from source-code level (program and data structure) to a higher level such as structure charts, data-flow models, and finally domain or requirements models.
- **Completeness:** how much of the system is recovered (for example, all the data-flow models or only some).
- **Interactivity:** how much the human takes part. Fully automatic tools give limited results; interactive tools combine tool output with the analyst's knowledge.
- **Directionality:** one-way (the information is given to the maintenance engineer who does the work) or two-way (information is fed to a re-engineering tool).

Level	Information recovered
Implementation level	Abstract syntax trees, symbol tables, call graphs, control-flow graphs
Structure level	Module dependencies, structure charts, data structures
Functional level	What each module does; data-flow diagrams
Domain (requirements) level	Business rules, use cases, requirements

16.4 Process of reverse engineering

- 1 **Collect information:** gather source code, executable files, any old documents, data files, and user knowledge.
- 2 **Study the code:** read the code to know the language, structure and size; run the program if possible.
- 3 **Extract structure:** find modules, functions, calls, data structures, files and database.
- 4 **Build models:** construct call graphs, structure charts, data-flow diagrams, ER diagrams, control-flow graphs.
- 5 **Add meaning:** work out purpose of each module, business rules, and user interface behaviour.

- 6 **Document:** create new design and specification documents and store them under configuration management.

Reverse engineering is commonly seen in three areas: **of data** (finding the internal data structures and database schema), **of processing** (what each part of the program does, and its algorithms), and **of user interfaces** (the screens, menus and how a user uses the system).

16.5 Tools

- Decompilers and disassemblers (convert executables to source or assembly).
- Source code analysers and cross-reference generators.
- Call-graph and control-flow generators, program slicers.
- Tools that draw structure charts, DFDs and class or ER diagrams from existing code or database.
- Debuggers and tracing tools for run-time behaviour.

16.6 Applications and limitations

Applications: software maintenance, re-engineering, migration, documentation recovery, interoperability, and security analysis. **Limitations:** it takes time; the original intent and business rules may not appear in code; automatically recovered models can be large and hard to read; and decompiling may have legal restrictions when software is protected by licence or copyright.

Exam Tip: Jun-24 and 108T both ask "What is reverse engineering?". Give: definition, direction (code to design to requirements), purpose, process in 5 or 6 steps, tools, and one line that it only analyses and does not change the system.

17. Software Re-engineering

17.1 Definition

Software re-engineering is the examination, analysis and alteration of an existing software system to **reconstitute it in a new, improved form**, followed by implementation of the new form. It is a combination of **reverse engineering** (to understand the system), **restructuring** (to clean it), and **forward engineering** (to build the improved system). The functionality of the system is kept; its quality, structure and technology are improved.

17.2 Objectives and need

- Improve maintainability of a system that is badly structured and difficult to change.
- Reduce high and growing maintenance cost.
- Move to newer technology, language, platform or database.
- Improve performance and reliability.
- Recover documentation.
- Extend the useful life of a valuable legacy system without the large cost and risk of writing it afresh.

Signs that a system needs re-engineering: frequent failures; code written in an obsolete language; poor and unstructured code; no or wrong documentation; the system is hard to change, and each change causes new faults; maintenance cost keeps rising; the hardware is obsolete; the system is still valuable to the business.

17.3 The re-engineering process (activities)

Activity	Description
1. Inventory analysis	Make a list of all the applications of the organisation with attributes such as age, size, business importance, maintenance effort, and number of defects. Rank them and choose the candidates for re-engineering where the benefit is greatest.
2. Document restructuring	Documentation is weak or missing. Choose between: do nothing if the system is stable and will not change; update documentation only when it is touched; or fully re-document if the system is critical.
3. Reverse engineering	Analyse the program to recover its design, data and procedural structure, and the specification (see Section 16).
4. Code restructuring	Analyse the code and convert unstructured or "spaghetti" code into structured code, without changing its behaviour. Re-compile, re-test and update the internal documentation.
5. Data restructuring	Analyse the current data architecture, clean up the data (data analysis, rationalising data names, and translating data to new formats or database). It often needs more effort than code restructuring.
6. Forward engineering	Use the recovered information to design and build the new system, using modern tools, methods and technology, and add improvements. Also called <i>renovation</i> or <i>reclamation</i> .

17.4 Cost-benefit considerations

Re-engineering is attractive when the cost of re-engineering plus the cost of maintaining the improved system is less than the cost of continuing to maintain the old system or of developing a new one. Organisations compare the options: scrap the system, continue maintaining as it is, re-engineer, or replace with a new system.

17.5 Advantages and limitations

Advantages	Limitations
Lower cost and risk than rewriting from scratch	Cannot make a very poor system perfect
Existing, proven functionality is preserved	Major architectural limits stay
Improved maintainability, documentation and performance	Needs skilled staff and good tools
Allows migration to new technology	Cost may be high for large systems; data restructuring is hard
Faster than a new development	Business knowledge in the old code may not be fully recovered

17.6 Re-engineering versus new development

The Jun-2025 paper asks to differentiate re-engineering from new development.

Basis	Re-engineering	New development
Starting point	An existing system and its code	Fresh user requirements; nothing exists
Requirements	Largely known from the old system; stable	Must be elicited, analysed and specified from the start
First activity	Reverse engineering to understand the old system	Requirements analysis
Cost	Usually lower	Usually higher
Time	Usually shorter	Usually longer
Risk	Lower, since proven functionality is reused	Higher, since everything is new
Documentation	Recovered, then updated	Produced in each phase
Aim	Improve quality, structure, and technology without changing function	Build new functionality
Flexibility	Limited by existing design	Freedom to choose design and technology
Typical use	Legacy systems with high maintenance cost	New business needs or products

17.7 Forward engineering, reverse engineering and re-engineering compared

Term	Direction	Result
Forward engineering	Requirements to design to code (the normal path)	A new system
Reverse engineering	Code to design to requirements	Information (models and documents) only
Re-engineering	Reverse engineering, then restructuring, then forward engineering	An improved version of the existing system

Exam Tip: Jun-24 Q7(c) asks for a short note on Software Re-engineering and Jul-23 asks "What is software re-engineering?". Write definition, need, the six-step process, advantages and the relation to reverse engineering. For Jun-25 add the comparison table with new development.

18. Configuration Management

18.1 Introduction and definition

During development and maintenance a software product keeps changing: requirements change, bugs are fixed, new versions are released, and many people work on the same files. Without control, this leads to confusion: nobody knows which version is the latest, a fix is lost, or the wrong files are shipped.

Software Configuration Management (SCM) is the discipline of **identifying, organising and controlling changes** to the software and its associated work products throughout the life cycle, so that the integrity and traceability of the product are maintained. It answers the questions: *What exists? What changed? Who changed it, when and why? Can we recreate any earlier version?*

18.2 Objectives

- Identify all the items that make up the software, and control them.
- Control and track changes in an orderly way.
- Make sure that the right version of each item is used and delivered.
- Allow any earlier version to be rebuilt.
- Support team work (many developers working on the same code).
- Give a full record of changes for audits and maintenance.

18.3 Basic terms

Term	Meaning
Software Configuration Item (SCI)	Any work product that is placed under configuration control: source code, SRS, design document, test cases, user manual, data files, build scripts, tools.
Configuration	The set of SCIs (with their versions) that make up a particular version of the product at one point in time.
Baseline	A specification or product that has been formally reviewed and agreed, and that serves as the basis for further development. It can be changed only through formal change control. (Examples: approved SRS, approved design, tested release.)
Version	A particular state of an SCI, created as the item evolves.
Revision	A new version that replaces the previous one (a correction or an enhancement).
Variant	A version that exists in parallel for a different environment or customer (for example Windows and Linux versions).
Release	A version made available to customers.
Repository (library)	The central store holding all SCIs and their history.
Check-out / Check-in	Taking a copy of an item from the repository to modify it, and putting the modified item back as a new version.

18.4 SCM activities

18.4.1 Configuration (item) identification

Decide which items will be controlled, give each a unique name (identifier) and version number, and define baselines. Identification uses a naming and version scheme, and records the relationships between items (for example, which code file implements which design module).

18.4.2 Version control

Version control manages the different versions and variants of each SCI. Using a repository and tools, it records every change, supports concurrent work (branching and merging), prevents one person overwriting another's change (locking or merging), and allows return to an earlier version (rollback).

- **Version numbering:** commonly *major.minor.patch* (for example 2.3.1). Major changes the first digit; added features change the second; bug fixes change the third.

- **Branching** creates a separate line of development (for a new feature or customer variant) and **merging** combines it back.

18.4.3 Change control

Change control is the formal procedure for handling requests to change a baselined item, so that change is made in a planned way and does not damage the product.

- 1 A **change request (CR)** is raised (by a user, customer or developer) describing the need.
- 2 The change is **evaluated** for its technical effect, cost, schedule and risk (impact analysis).
- 3 The **Change Control Board (CCB)**, a group of senior technical and management people, approves, rejects or defers the change.
- 4 If approved, an **engineering change order (ECO)** is issued, and the developer checks out the item, makes the change, and tests it.
- 5 The change is **reviewed and audited**, checked in as a new version, and placed under version control.
- 6 The new baseline or release is created, and the change is reported to all the people concerned.

18.4.4 Configuration audit

Audit checks that the change has been made correctly and that the product matches what is in the records. Two kinds of check are used: a **formal technical review** (is the change technically correct, were the standards followed, have the related items been updated?) and a **software configuration audit** (was the change made as specified, were all related SCIs updated, did the change follow the process, and are the records complete?).

18.4.5 Configuration status reporting (accounting)

Records and reports what happened: which items exist and their versions, what changes were made, when, by whom and why, and what is the status of each change request. It keeps everyone informed, and avoids duplicated work and confusion.

18.5 SCM process summary

Step	Question it answers
Identification	What are the items, and how do we name them?
Version control	How do we manage the versions of each item?
Change control	How do we decide and make changes?
Audit	Was it done correctly?
Status reporting	Who needs to know what has changed?

18.6 SCM tools

- **Version control systems:** Git (distributed, most widely used), Subversion (SVN), CVS, Mercurial, Perforce, IBM ClearCase.
- **Hosting platforms:** GitHub, GitLab, Bitbucket.

- **Build and integration tools:** Make, Maven, Gradle, Jenkins (continuous integration).
- **Change and issue tracking:** Jira, Bugzilla.

18.7 Benefits of SCM

- Orderly control of change, so fewer mistakes and lost work.
- Traceability: it is known who changed what and why.
- Many developers can work in parallel without conflicts.
- Any previous version can be rebuilt, and faulty changes can be rolled back.
- Consistent builds and releases; support for audits and quality standards.
- Better quality and lower cost of maintenance.

Exam Tip: Jul-23 asks "What do you understand by Configuration Management?" (6.5 marks) and Jun-24 Q7(b) asks for a short note. Write: why needed, definition, SCI and baseline, the five activities, change control steps, tools, and benefits.

Part D: Exam Preparation

19. Previous-Year Questions with Answer Outlines

Every question from your question bank that belongs to Unit III (parts 1 to 3) is listed below with the sections that answer it and a suggested structure.

Question (paper, marks)	Where to read	How to answer
Explain different types of coupling (Jul-23 Q6a, 6)	Sec 4	Definition, ordered scale, six types with one-line example each, best and worst.
Define module coupling and explain types (Jun-25 Q7a, 5)	Sec 4	As above, in a shorter form.
Define coupling, types, which is best and why (108T Q7a, 5)	Sec 4.3.6	Add the "Exam Answer" box on data coupling.
Explain different types of cohesion (Jul-23 Q7a, 6)	Sec 3	Definition, seven types in order, example for each, table.
What is a software module and cohesion? Types (Jun-24 Q6, 12.5)	Sec 2.1, 3	Module definition and benefits, cohesion definition, full detail on seven types with code, advantages of high cohesion, coupling link.
What is design? Conceptual vs technical design (Jun-25 Q6b, 5)	Sec 1	Definition of design, then the comparison table. (Outside syllabus list but asked.)
SQA and its activities (Jul-23 Q6b, 6.5)	Sec 8	Definition, objectives, SQA group, activity table, SQA plan.
What is software review and software inspection? (Jul-23 Q1d, 5)	Sec 10, 11	Definition of each, Fagan phases, comparison table, benefits.
Short note: TQM (Jun-24 Q7a); What do you mean by TQM? (108T Q1h, 5)	Sec 9	Definition, principles, PDCA, benefits, TQM vs SQA.
Key components of quality management (Jun-25 Q6a, 5)	Sec 7	Planning, QA, QC, improvement; QA vs QC table.
What is quality in software? How do we determine SQA? Various SQA activities (108T Q6, 10)	Sec 6, 8	Quality definitions, McCall factors, how quality is determined, SQA definition, activities.
What is software re-engineering? (Jul-23 Q1b, 5); Short note (Jun-24 Q7c)	Sec 17	Definition, need, six process steps, advantages.
Re-engineering vs new development (Jun-25 Q8b, 5)	Sec 17.6	Short definition then the comparison table.
What is reverse engineering? (Jun-24 Q1d; 108T Q1b, 5)	Sec 16	Definition, purpose, process, tools, note that it does not change the software.
Configuration management (Jul-23 Q7b, 6.5; Jun-24 Q7b)	Sec 18	Need, definition, SCI, baseline, five activities, change control, tools.

20. Likely Questions from Topics Not Yet Asked

The following syllabus topics have no direct question in the four papers you have, so they are good candidates for future papers.

- Write a short note on structured programming and structured coding techniques. (Sec 12)
- What is a coding standard? Differentiate between coding standards and coding guidelines. (Sec 14)
- Explain coding style and give the features of good coding style. (Sec 13)
- Explain internal and external documentation and documentation guidelines. (Sec 15)
- Differentiate between a walkthrough, a review and an inspection. (Sec 11.6)
- Explain the cost of quality and McCall quality factors. (Sec 6)
- Explain the difference between QA and QC. (Sec 7)
- Explain baseline and SCI, and the role of the Change Control Board. (Sec 18)
- Explain fan-in and fan-out. (Sec 2.4)

21. One-Page Revision Summary

Topic	Remember
Cohesion (worst to best)	Coincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional
Coupling (worst to best)	Content, Common, External, Control, Stamp, Data
Good design	High cohesion, low coupling; low fan-out, high fan-in
Quality factors (McCall)	Operation: correctness, reliability, efficiency, integrity, usability. Revision: maintainability, flexibility, testability. Transition: portability, reusability, interoperability
Quality management	Planning, assurance (process, prevent), control (product, detect), improvement
SQA	Planned activities across the life cycle: plan, review, audit, record deviations, report, metrics
TQM	Whole organisation, customer focus, continuous improvement (PDCA), prevention
Review vs inspection	Inspection is formal: moderator, checklist, six phases, metrics
Fagan phases	Planning, Overview, Preparation, Meeting, Rework, Follow-up
Structured coding	Sequence, selection, iteration; single entry and exit; no GOTO (Bohm-Jacopini)
Standards vs guidelines	Standards mandatory and checked; guidelines are advice
Documentation	Internal (in code) and external (SRS, design, manuals, test plans)
Reverse engineering	Code to design to requirements; recovers information, changes nothing
Re-engineering steps	Inventory analysis, document restructuring, reverse engineering, code restructuring, data restructuring, forward engineering
SCM activities	Identification, version control, change control, audit, status reporting

Appendix: Asked in Papers but Outside the Syllabus List

Two questions in your papers fall in this unit but are not named in the syllabus you supplied. Short answers are given so you are not caught out.

A. CMM versus ISO 9000 (108T Q7b, 5 marks)

CMM (Capability Maturity Model) was developed by the Software Engineering Institute (SEI) at Carnegie Mellon University. It assesses the **maturity of the software process** of an organisation on a scale of five levels.

Level	Name	Characteristic
1	Initial	Ad hoc, chaotic; success depends on individual effort.
2	Repeatable (Managed)	Basic project management: planning, tracking, requirements and configuration management; earlier success can be repeated.
3	Defined	Organisation-wide standard process, documented and followed; training programmes.
4	Managed (Quantitatively managed)	Process and product are measured and controlled with metrics.
5	Optimising	Continuous process improvement from measurement and new ideas.

ISO 9000 is a family of international standards for **quality management systems**, applicable to any industry. ISO 9001 is the standard against which an organisation is certified.

Basis	CMM	ISO 9000
Scope	Software organisations only	Any organisation or industry (generic)
Issued by	Software Engineering Institute (SEI)	International Organization for Standardization
Result	Maturity level from 1 to 5	Certified or not certified (pass/fail)
Focus	Process improvement, continuous growth	Meeting the minimum quality system requirements
Nature	Detailed, software-specific key process areas	Broad, general requirements of a quality system
Improvement	Built-in path to higher levels	Certification maintained through periodic audit

B. Object-oriented terms (Jun-25 Q7b, 5 marks)

Term	Definition
Object	A real-world entity or run-time instance that has state (data/attributes) and behaviour (methods). Example: a particular student.

Term	Definition
Class	A blueprint or template that defines the attributes and methods common to a set of objects. Objects are instances of a class.
Message	A request sent from one object to another to invoke one of its methods; the way objects communicate.
Abstraction	Showing only the essential features and hiding the background details; for example a class exposes what an operation does and not how.
Inheritance	A mechanism by which a new (derived) class acquires the attributes and methods of an existing (base) class, supporting reuse and an "is-a" hierarchy.
Polymorphism	The ability of the same message or operation to behave differently for different objects (for example overloading and overriding); "many forms".

Final Note: Good luck with the exam. Practise drawing the cohesion and coupling scales, and the re-engineering and Fagan inspection flows, because diagrams add marks.